UNIVERSITE DE YAOUNDE I
****
ECOLE NATIONALE SUPERIEURE POLYTECHNIQUE
****
DEPARTEMENT DE GENIE INFORMATIQUE

UNIVERSITY OF YAOUNDE I
****
NATIONAL ADVANCED SCHOOL OF ENGINEERING
****
DEPARTMENT OF COMPUTER ENGINEERING

MEMORY VIRTUALIZATION OVERHEAD MITIGATION USING CONTIGUOUS MEMORY VIRTUAL MACHINES

Case study : Xen virtualization system

End of course dissertation/Master of Engineering

Presented and defended by

YUHALA PETERSON JR. DOHBILA

In partial fulfilment of the requirements for the degree of:

Master of Engineering in Computer Science

Under the supervision of:

ALAIN TCHANA, PROFESSOR, UNIVERSITY OF NICE - SOPHIA ANTIPOLIS

BORIS TEABE, POSTDOCTORAL RESEARCHER, TOULOUSE COMPUTER SCIENCE RESEARCH INSTITUTE

Committee Members:

President: CLAUDE TANGHA, PROFESSOR, UNIVERSITY OF YAOUNDE I

Reporter 1: ALAIN TCHANA, PROFESSOR, UNIVERSITY OF NICE - SOPHIA ANTIPOLIS

Reporter 2: BERNABÉ BATCHAKUI, SENIOR LECTURER, UNIVESITY OF YAOUNDE I

Examiner: GEORGES KOUAMOU, SENIOR LECTURER, UNIVESITY OF YAOUNDE I

Academic year 2017-2018
Defended on the 14th of September 2018

MEMORY VIRTUALIZATION OVERHEAD MITIGATION USING CONTIGUOUS MEMORY VIRTUAL MACHINES

Case study : Xen virtualization system

End of course dissertation/Master of Engineering

Presented and defended by

YUHALA PETERSON JR. DOHBILA

In partial fulfilment of the requirements for the degree of:

Master of Engineering in Computer Science

Academic year 2017-2018
Defended on the 14th of September 2018

Dedication

To my beloved parents

" True wisdom comes to each of us when we realize how little we understand about life, ourselves, and the world around us."

Socrates

ACKNOWLEDGEMENTS

This work is the culmination of the efforts and contributions of several people who stood by my side during my tenure as an engineering student. I would like to express my profound gratitude and appreciation to:

- **Claude TANGHA**, Professor at UY1, for accepting to preside over the jury;
- **Georges KOUAMOU**, Senior Lecturer at UY1, for agreeing to examine this work and for his remarkable dedication to teaching;
- **Alain TCHANA**, Professor at the University of Nice - Sophia Antipolis and my supervisor, for giving me the unique opportunity to do an internship at IRIT and for his expertise, inspiring guidance, availability and immense contributions towards the accomplishment of this work;
- **Bernabé BATCHAKUI**, Senior Lecturer at UY1 and my academic supervisor, for all his guidance and advice;
- **Daniel HAGIMONT**, Professor at INPT and head of the SEPIA team, for the enormous contributions he made towards the accomplishment of my work at the IRIT laboratory ;
- **Boris TEABE**, postdoctoral researcher at the IRIT laboratory and one of my supervisors, for his constant encouragement and the pragmatic suggestions he made during the entire tenure of my final internship;
- **Michel DAYDÉ**, Professor at INPT and Director of the IRIT laboratory, for permitting me to do an internship at the IRIT laboratory.
- The entire teaching staff of ENSPY, especially those of the Computer Engineering department, under the guidance of Pr. **Thomas BOUETOU**, who instilled in me a great value for education.
- My parents **Peter YUHALA** and **Beatrice FOTACHWI**. Their love, affection, and unswerving encouragement gave me solace and made me strong in my commitment to achieve all my educational goals;
- My siblings **Nadia NJOME, Kathya YUHALA, Julia YUHALA** and **Darren YUHALA** for all their love and kindness;
- **Djob MVONDO, Kevin JIOKENG, Lavoisier WAPET, Stella N'DONGA, Tu DINH, Bao BUI, Priscille TETCHOU, Grégoire TODESCHI, Vlad NITU, Mathieu BACOU**, my lab mates, for their unwavering support;

- The entire class of 2018, especially **Vanessa EBOLE, Rahi'matou DAOUDA, Nadia DJONGOUE, Boris FOTSA, Ruben UM-GWET and Raoul DZOUKOU** for their support and encouragement, and the good times spent together.
- To all those who in one way or another contributed in the completion of this work.

Abbreviations

AMD Advanced Micro Devices.

CPU Central Processing Unit.

gPA Guest Physical Address.

gVA Guest Virtual Address.

hPA Host Physical Address.

HVM Hardware Virtual Machine.

I/O Input/Output.

INPT National Polytechnic Institute of Toulouse.

IRIT Toulouse Computer Science Research Institute.

KVM Kernel-based Virtual Machine.

MMU Memory Management Unit.

OS Operating System.

PTE Page Table Entry.

PV Paravirtualization.

RAM Random Access Memory.

TLB Translation Lookaside Buffer.

UY1 University of Yaounde I.

VM Virtual Machine.

VMCS Virtual Machine Control Structure.

VMM Virtual Machine Monitor.

GLOSSARY

context switch The switching of the CPU from one process or thread to another.

fragmentation A process where memory is broken up into lots of little pieces, none of which is big enough to satisfy the next memory requirement.

hypercall A software trap from a virtual machine to the hypervisor.

linux An open-source operating system modelled on UNIX.

OS System software that manages computer hardware and software resources.

overhead The cost of carrying out a specific operation.

page fault An interrupt that occurs when a program requests data that is not currently in real memory.

page walk The process of going through a page table to obtain the page table entry corresponding to a virtual address.

physical address The location of a memory cell in main memory (RAM).

radix tree A data structure that represents a space-optimized trie in which each node that is the only child is merged with its parent.

server consolidation The use of a physical server to accommodate one or more server applications or user instances.

source code Text listing of commands to be compiled or assembled into an executable computer program.

virtual address An address in a process's virtual address space.

vm trap A synchronous interrupt typically caused by an exceptional condition.

vmm Computer software, firmware or hardware that creates and runs virtual machines.

x86 A family of backward-compatible instruction set architectures based on the Intel 8086 CPU and its Intel 8088 variant

.

Abstract

Virtual machines are playing an increasing role in server consolidation, security, and fault tolerance as computing systems migrate to shared resources in the cloud. Since the software stack accesses data using virtual addresses, fast address translation is crucial for efficient data-centric computation and for providing the benefits of virtualization to a wide range of applications. Unfortunately, the overhead of virtualizing memory is not universally low. This overhead comes from costly hypervisor interventions and from translation lookaside buffer (TLB) misses which require the hardware to perform a two dimensional page walk, which may require up to 24 memory references on x86-64 architecture, rather than a native one dimensional page walk which could incur a maximum of 4 memory references. In this work, we propose a new mechanism of address translation in virtualized systems which is based on contiguous-memory virtual machines, and whose overall overhead is theoretically equivalent to that in native environments. Our basic idea is to map virtual machine physical address space to contiguous machine memory which would eliminate the need for a second page table during address translation. We developed a simulator to replay virtual machine traces from several well known data centers so as to demonstrate the feasibility of allocating contiguous machine memory to virtual machines using existing placement algorithms. We then implemented the first phase of our solution by modifying the Xen hypervisor so it takes into account contiguous memory allocations for newly created virtual machines.

Keywords: Virtualization, Virtual Memory, Translation Lookaside Buffer, Contiguous Memory.

Résumé

Les machines virtuelles jouent un rôle croissant dans la consolidation des serveurs, la sécurité et la tolérance aux pannes comme les systèmes informatiques migrent vers des ressources partagées dans le cloud. Étant donné que la pile logicielle accède aux données à travers les adresses virtuelles, la translation rapide d'adresses est essentielle pour les calculs centrés sur les données et pour offrir les avantages de la virtualisation à une grande variété d'applications. Malheureusement, le coût de la virtualisation de la mémoire n'est pas faible. Ce coût provient des interventions très coûteuses de l'hyperviseur et des échecs du cache TLB (Translation Lookaside Buffer) qui nécessitent que le matériel effectue un parcours de page bidimensionnel, ce qui peut nécessiter jusqu'à 24 accès mémoire dans l'architecture x86-64, plutôt qu'un parcours de page unidimensionnel dans les systèmes natifs, qui nécessite un maximum de 4 accès mémoire. Dans ce travail, nous proposons un nouveau mécanisme de translation d'adresses dans les systèmes virtualisés basé sur des machines virtuelles à mémoire contiguë et dont le coût globale de la translation est théoriquement équivalente à celle des systèmes natifs. Notre idée de base est d'établir une correspondance entre l'espace d'adressage physique de la machine virtuelle et un bout contiguë de la mémoire physique, ce qui éliminerait le besoin d'une deuxième table de pages lors de la translation des adresses. Nous avons développé un simulateur pour rejouer les traces de machines virtuelles de plusieurs data centers bien connus afin de démontrer la faisabilité d'allouer de la mémoire contiguë aux machines virtuelles en utilisant les algorithmes de placement existants. Nous avons ensuite implémenté la première phase de notre solution en modifiant l'hyperviseur Xen pour prendre en compte les allocations de mémoire contiguës pour les machines virtuelles nouvellement créées.

Mot clés: Virtualisation, Mémoire Virtuelle, Translation Lookaside Buffer, Mémoire Contiguë.

List of Tables

Table **Page**

2.1 Trade-offs provided by shadow and nested paging as compared to base native 21

3.1 Trade-offs provided by our solution 28

4.1 Physical server composition used for simulation 37

4.2 CERIT-SC cluster characteristics 38

List of Figures

Figure		Page
1.1	Virtualization system	4
1.2	Full virtualization architecture	5
1.3	Paravirtualization architecture	5
1.4	OS-level virtualization architecture	6
1.5	Memory architecture in a native system	8
1.6	Address translation in native systems	9
1.7	Page walking mechanism in x86_32 architecture	10
1.8	Memory architecture in a virtualized system	11
1.9	Address translation in virtualized systems	11
1.10	Xen virtualization system	13
2.1	x86_64 native $VA \rightarrow PA$ translation	16
2.2	x86_64 native $gVA \rightarrow hPA$ translation	17
2.3	Native applications, 4KB pages	18
2.4	Applications running in VM, g4KB/h4KB pages	18
2.5	Shadow paging	19
2.6	Pseudocode for native, nested, and shadow page walk state machines.	21
3.1	Mapping guest physical address space to a contiguous portion of RAM	24
3.2	Our translation mechanism	25
3.3	General architecture of our model	25
4.1	The Xen Heap Structure	34
4.2	Amount of memory per VM in GB	38
4.3	Number of virtual CPU cores per VM	38
4.4	Percentage of contiguous-memory VMs in the Microsoft Azure dataset	39
4.5	Percentage of contiguous-memory VMs in the CERIT-SC dataset	40
4.6	Percentage of contiguous-memory VMs in the Bitbrains dataset (fastStorage)	40
4.7	Consolidation Rates for the different placement algorithms	41

Table of Contents

Dedication i

Acknowledgements ii

Abbreviations iii

Glossary v

Abstract vi

Résumé vii

List of Tables viii

List of Figures ix

Table of Contents xi

Introduction 1

1 Background 3

1.1 Server virtualization 4

1.1.1 Server virtualization techniques 4

1.1.2 Advantages of server virtualization 6

1.2 Memory Virtualization 8

1.2.1 Virtual Memory and Paging 8

1.2.2 Address translation in native systems 9

1.2.3 Address translation in virtualized systems 10

1.3 Xen hypervisor 12

1.3.1 The Xen Architecture 12

2 Problem statement and state of the art 15

2.1 Problem Statement 16
2.1.1 Address translation overheads in x86_64 16
2.2 State of the art address translation techniques in virtualized systems 18
2.2.1 Hardware-based Nested Paging 18
2.2.2 Hypervisor-enabled Shadow paging 19
2.2.3 Other Related Work 21

3 Contribution **23**
3.1 Proposed Model 24
3.1.1 Basic Idea 24
3.1.2 Detailed description of our solution 25
3.2 Reducing Fragmentation 26
3.2.1 Memory Compaction 26
3.2.2 Reclaiming I/O gap memory 27
3.3 Advantages of our mechanism 27
3.3.1 Comparison of trade-offs 28

4 Implementation and Evaluation **29**
4.1 Implementation 30
4.1.1 Simulation 30
4.1.2 Hypervisor modification 32
4.1.3 Guest OS 35
4.1.4 Development tools 36
4.2 Evaluation 36
4.2.1 Evaluation Methodology 36
4.2.2 Experimental Setup 39
4.2.3 Simulation Results 39
4.2.4 Analysis of Results 41

Conclusion **43**

References **44**

Annex **47**

INTRODUCTION

CONTEXT

Virtualization forms the foundation of our cloud infrastructure. It provides benefits including security, isolation, server consolidation[1] and fault tolerance, and enables easier scalability of applications and higher system utilization by abstracting the underlying hardware resources. These benefits became achievable because various hardware and software advances have substantially reduced the cost of virtualization [12]. Nonetheless, despite hardware and software acceleration [12] the overhead of virtualizing memory can still be very high.

One of the largest contributors to performance overhead in virtualized environments is **memory virtualization**. An application executing on a guest operating system (OS) uses guest virtual addresses (gVA) that need to be translated to host physical addresses (hPA). The guest OS executes in guest physical address (gPA) space. To fully virtualize memory therefore, two levels of address translation are used: **gVA→gPA** : guest virtual address to guest physical address translation via a per-process guest page table (gPT) and **gPA→hPA** : guest physical address to host physical address translation via a per-VM host page table (hPT).

There are two state-of-the-art techniques to virtualize memory which provide different tradeoffs. The first is **nested paging** [6], which is a hardware technique whereby the host OS uses the gPT and the hPT simultaneously in order to get the final **gVA→hPA** translation. In case of a **TLB miss** , the hardware performs a long-latency two dimensional (2D) **page walk** which could require up to 24 memory accesses [6] in x86-64 systems. The second technique is called **shadow paging** [27], which requires the VMM to build a **shadow page table** (sPT) for each VM. The shadow page table contains **gVA→hPA** translations and in case of a TLB miss, the hardware performs a fast native one dimensional (1D) page walk. However, page table updates on context switches require the VMM to perform substantial work to keep the page tables consistent [1].

PROBLEMATIC

With current hardware and software, the overheads of virtualizing memory are hard to minimize because VMs use exclusively one technique or the other. Nested paging can require up to 24

[1] An approach to the efficient usage of computer server resources so as to minimize the total number of servers used.

memory references as opposed to a native 4 memory references, and shadow paging requires slow page table updates as against fast and direct page table updates in base native environments. We are therefore faced with an interesting challenge:

Questions

- How can we virtualize memory and still preserve base native performance ?

OBJECTIVES

The main goal of this work is to establish an **address translation mechanism** in virtualized systems that would present **zero overhead** vis-à-vis base native. Our technique of address translation is based on **contiguous memory** VMs and it should enable us to:

1. nullify address translation overhead in virtualized systems,
2. enhance memory isolation knowing that the hypervisor has minimal interference,
3. enforce memory contiguity in VMs throughout their lifetime.

To attain these objectives, we began by analyzing the limitations of state-of-the-art techniques of address translation in virtualized systems, and did a theoretical comparison of our solution with the aforementioned techniques. We then carried out a series of simulations so as to do a proof of concept of our idea, after which we implemented and evaluated the performance of our model.

OUTLINE

The rest of this document is structured as follows:

- **Background** which presents a review of current literature on server virtualization and memory virtualization so as to provide a larger context into which our work will fit;
- **Problem statement and state-of-the-art** wherein we present the claim that outlines the problem addressed by our work, and the different techniques which have been proposed as solutions to the problem of address translation in virtualized systems ;
- **Contribution** which provides a detailed theoretical description of our solution, as well as a theoretical comparison of our model with existing address translation mechanisms;
- **Implementation and Evaluation** wherein we discuss the implementation details of our model, the software tools used, and simulation results obtained.

CHAPTER 1

BACKGROUND

In this chapter we discuss key concepts in virtualization technology such as server virtualization, virtualization techniques and the fundamentals of memory virtualization. The outline of this chapter is as follows:

Contents

1.1 Server virtualization **4**

1.1.1 Server virtualization techniques 4

1.1.2 Advantages of server virtualization 6

1.2 Memory Virtualization **8**

1.2.1 Virtual Memory and Paging 8

1.2.2 Address translation in native systems 9

1.2.3 Address translation in virtualized systems 10

1.3 Xen hypervisor **12**

1.3.1 The Xen Architecture 12

1.1 Server virtualization

Server virtualization is a technique which allows us to create an abstract layer of system resources and thus hide the complexity of hardware and software working environments [16, 21]. Virtualization provides hardware independence, isolation of the guest operating system and encapsulation of the entire virtual machine. To carry out server virtualization, we need a virtualization system or **hypervisor**.

A hypervisor or Virtual Machine Monitor (VMM) [16] is a software layer that can monitor and virtualize the resources of the host operating system[1]. Typically hypervisors can be subdivided into two categories : Type 1 and Type 2, based on the level of implementation. Type 1 hypervisors interact directly with the hardware (bare metal) and thus do not require a host OS to operate. Hypervisors such as KVM, Hyper-V and Citrix XenServer are identified in this category. Type 2 hypervisors on the other hand run on top of a host operating system. Hypervisors such as VirtualBox and VMware Workstation fall into this category. Figure 1.1 gives us a pictorial representation of a virtualization system. In the next section we would discuss different server virtualization techniques.

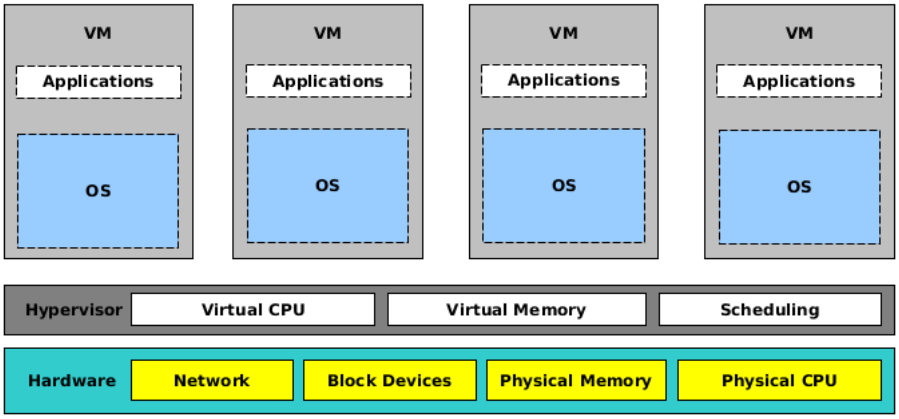

Figure 1.1: Virtualization system

1.1.1 Server virtualization techniques

In general, most server virtualization strategies fall into one of four major categories:

[1]The OS that interacts directly with the hardware

1.1.1.1 Full virtualization

This is sometimes referred to as **hardware emulation**. Here an unmodified operating system is run using a hypervisor which traps[2] and safely translates and executes privileged instructions on-the-fly. The VMM provides a virtual BIOS, virtual devices and virtual memory to the guest OS, which is not aware it is being virtualized. Full virtualization provides the best security and isolation for virtual machines and provides easy migration as the same guest OS instance can run on virtualized or native hardware. **Microsoft Virtual Server and VMware ESXi** use this technique. Figure 1.2 presents the full virtualization architecture.

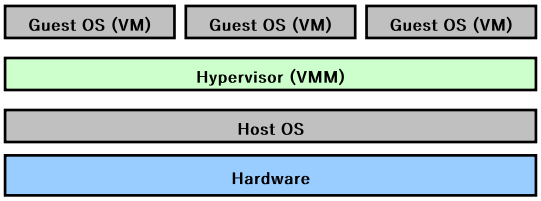

Figure 1.2: Full virtualization architecture
Source: [24]

1.1.1.2 Paravirtualization

This is a virtualization technique wherein the guest OS kernel is modified in order to replace non-virtualizable instructions with hypercalls[3] which can communicate directly with the virtualization layer i.e. the hypervisor. This reduces the need for privileged instructions that are typically responsible for major performance penalties in full virtualization. However, the deep kernel modifications required by paravirtualization could introduce support and maintainability issues in production environments. **Xen, VMWare and ESX Server** use this technique. The following figure illustrates the paravirtualization architecture.

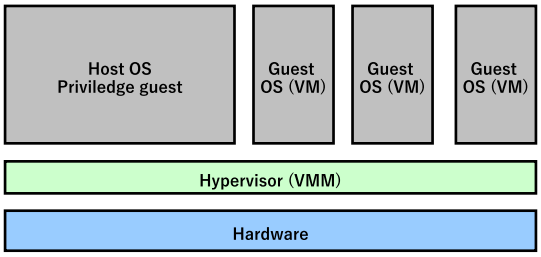

Figure 1.3: Paravirtualization architecture
Source: [24]

[2] A synchronous interrupt typically caused by an exceptional condition.
[3] Synchronous calls from a guest OS to the hypervisor.

1.1.1.3 Hardware Assisted virtualization

This is a particular case of server virtualization in which the hardware is designed in such a way as to facilitate virtualization. For example, hardware vendors **Intel** and **AMD** introduced first generation enhancments such as **Intel Virtualization Technology (VTx)** and **AMD-V** which both target privileged instructions with a new CPU execution mode feature that allows the VMM to run in a new root mode below ring 0, in such a way that privileged and sensitive calls are set to automatically trap to the hypervisor, removing the need for either binary translation[4] or paravirtualization. Hypervisors such as **VMWare, vCenter and Xen** support hardware assisted virtualization.

1.1.1.4 Operating System - level Virtualization

This is a technique of virtualization which, unlike the previously mentioned techniques, does not rely on a hypervisor. In this case, the OS is modified to securely isolate multiple instances of an OS within a single host machine. The guest OS instances are often referred to as **virtual private servers (VPS)**. The main advantage of OS-level virtualization lies in performance, as there is no hypervisor-instruction trapping, leading to system performance of near-native speeds. The primary disadvantage is that all VPS instances share a single kernel. Thus, if the kernel is compromised, all VPS instances are compromised. However, the advantage of having a single kernel is that fewer resources are consumed leading to less overhead when compared to multiple kernel systems. This type of virtualization is offered by **Linux-Vserver, Chroot, BSD Jail LXC (Linux Container)**. Figure 1.4 illustrates OS-level virtualization.

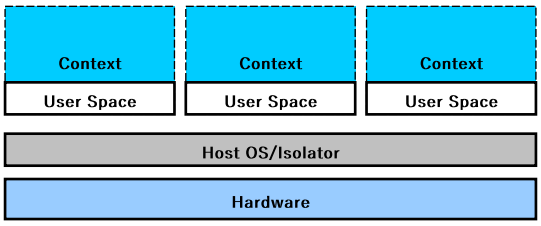

Figure 1.4: OS-level virtualization architecture
Source: [24]

1.1.2 Advantages of server virtualization

The basic motivation for virtualization is the same as that for multitasking operating systems; computers have more processing power than one task needs. The first computers were built to do one task. The second generation was programmable; these computers could do one task, and

[4]Translation of sensitive guest OS instructions into privileged instructions.

then do another task. Eventually, the hardware became fast enough that a machine could do one task and still have spare resources.

A lot of organizations are now finding that they have a lot of servers all doing single tasks, or clusters of related tasks. Virtualization allows a number of virtual servers to be consolidated into a single physical machine, without losing the security gained by having completely isolated environments. To provide an explicit explanation of why more companies and organizations are switching to virtualization, here are some key benefits server virtualization can offer:

Security and Isolation: By compartmentalizing environments with different security requirements in different virtual machines, one can select the guest OS and tools that are more appropriate for each environment. For example, we may want to run an Apache web server on top of a Linux guest OS and a backend MySQL server on top of a guest Windows XP OS, all in the same physical platform. A security attack on one virtual machine does not compromise the others because of their isolation.

Reduced Costs: It is possible to achieve cost reductions by consolidating smaller servers into more powerful servers. Cost reductions stem from hardware cost reductions (economies of scale seen in faster servers), operations cost reductions in terms of personnel, floor space, and software licenses. VMWare cites an overall cost reduction ranging from 29 to 64% [26].

Load Balancing: Since the software state of an entire virtual machine is completely encapsulated by the VMM, it is relatively easy to migrate virtual machines to other platforms in order to improve server performance through load balancing [25].

Adaptability to Workload Variations: Changes in workload intensity levels can be easily taken care of by shifting resources and priority allocations among virtual machines. Autonomic computing-based resource allocation techniques [17] can be used to dynamically move processors from one virtual machine to another.

Reliability and Availability: Server virtualization allows migration[5] of virtual machines between physical servers. In case of failure of one server, the virtual machine is automatically migrated to another server, thereby greatly increasing availability of services.

Legacy Applications: If an organization decides to migrate to a different OS, it is possible to continue to run legacy applications on the previous OS running as a guest OS within a virtual machine.

[5]The process of moving a running VM from one physical hardware environment to another.

Reduced licensing costs. One operating system license per server is required. However, virtualization sometimes allows you to take advantage of license packs covering the OS of the physical server and its virtual machines. For existing virtual machines, it will not be necessary to repay the license fees when the physical server on which the virtual machines run is changed. This also applies to applications running within these virtual machines.

1.2 Memory Virtualization

Memory virtualization is a very critical component in virtualized systems and it involves managing physical/machine memory and dynamically allocating it to virtual machines. Memory virtualization in virtualized systems is very similar to the virtual memory support provided by modern operating systems. To get a better grip of the overall concept of memory virtualization, let us look start by discussing virtual memory and paging.

1.2.1 Virtual Memory and Paging

Virtual memory is a crucial abstraction we need to efficiently use physical memory in today's computer systems. It delivers benefits such as security due to process isolation, allows for very effective multiprogramming and relieves the user of the unnecessarily tight constraints of main memory. **Paging** is a memory management scheme by which a computer stores and retrieves data from secondary storage for use in main memory. Each process in an OS has a very large virtual address space managed at a granularity of pages, typically 4 kilobytes (KB) in size. Figure 1.5 illustrates the memory architecture in an operating system.

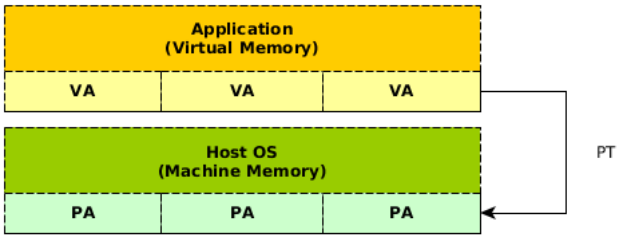

Figure 1.5: Memory architecture in a native system

The operating system uses a per-process **page table** to store **virtual to physical address** mappings. All modern x86 processors include a **memory management unit (MMU)** and a **translation lookaside buffer (TLB)** to optimize virtual memory performance. The TLB is a high speed CPU cache that stores the **page table entries** for recently accessed virtual addresses. In x86_64, there is a per-core hardware register called the **control register 3 (CR3)**, which stores the physical address that points to the top-level of the corresponding page table and its value is the same for a given process.

1.2.2 Address translation in native systems

From the above memory architecture, we understand that to fully virtualize memory in a native system, only one level of address translation is required: **VA→PA**, as illustrated in Figure 1.6.

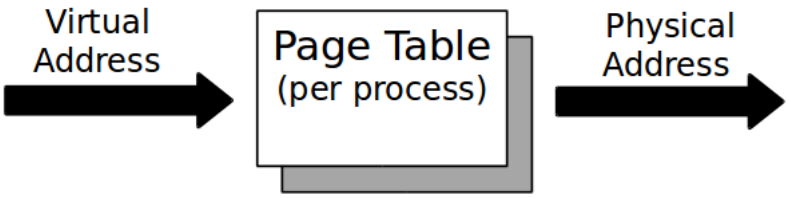

Figure 1.6: Address translation in native systems

Given a virtual address, the MMU first examines the TLB. If the desired page table entry is present (**TLB hit**), then the **frame number** is retrieved and the real address is formed. If the desired page table entry is not found (**TLB miss**), the MMU reads the value in the **control register 3 (CR3)** which represents the address of the **page global directory**[6] of the current process. The MMU then examines the corresponding page table in a process called a **page walk**, after which the page table entry corresponding to the virtual address is obtained. If the "present bit" is set (1), then the page is in main memory (RAM memory), and the MMU can retrieve the frame number from the page table entry to form the real address. The processor then updates the TLB to include this new page table entry. If the "present bit" is not set (0), the desired page is not in main memory and a memory access fault, called a **page fault**, is issued. At this point, we leave the realm of hardware and invoke the OS, which loads the needed page into main memory and updates the page table [20].

1.2.2.1 How a page walk is implemented

As we mentioned above, a page walk is simply a process by which the processor examines the page table of a process so as to obtain the page table entry corresponding to the virtual address. On x86_64 architecture, the page table consists of 4 levels: 2

- Level 0 : Page Map Level 4 (PML4)
- Level 1 : Page Directory Pointer (PDP)
- Level 2 : Page Directory (PD)
- Level 3 : Page Table Entry (PTE)

On x86_32 architecture, we have only 2 levels:

[6] The top level of a page table

- Level 0 : Page Directory (PD)
- Level 1 : Page Table Entry (PTE)

Page walks on both architectures are similar but for the sake of simplicity, we would illustrate the page walking mechanism using a 2 level page table.

Given a 32 bit virtual address, the address of the page directory (the topmost level) is found by looking into the CR3 register. The MMU then uses the first 10 bits of the virtual address as an index into the page directory. The value at that index gives us the address of the page table. The next 10 bits of the virtual address are then used as an index into the page table. The value at this index represents the address of the actual physical page containing the process data/code. The last 12 bits of the virtual address are used as an offset into the 4KB physical page where the process will read/write the data. The figure below illustrates the page walking mechanism in a 2-level page table.

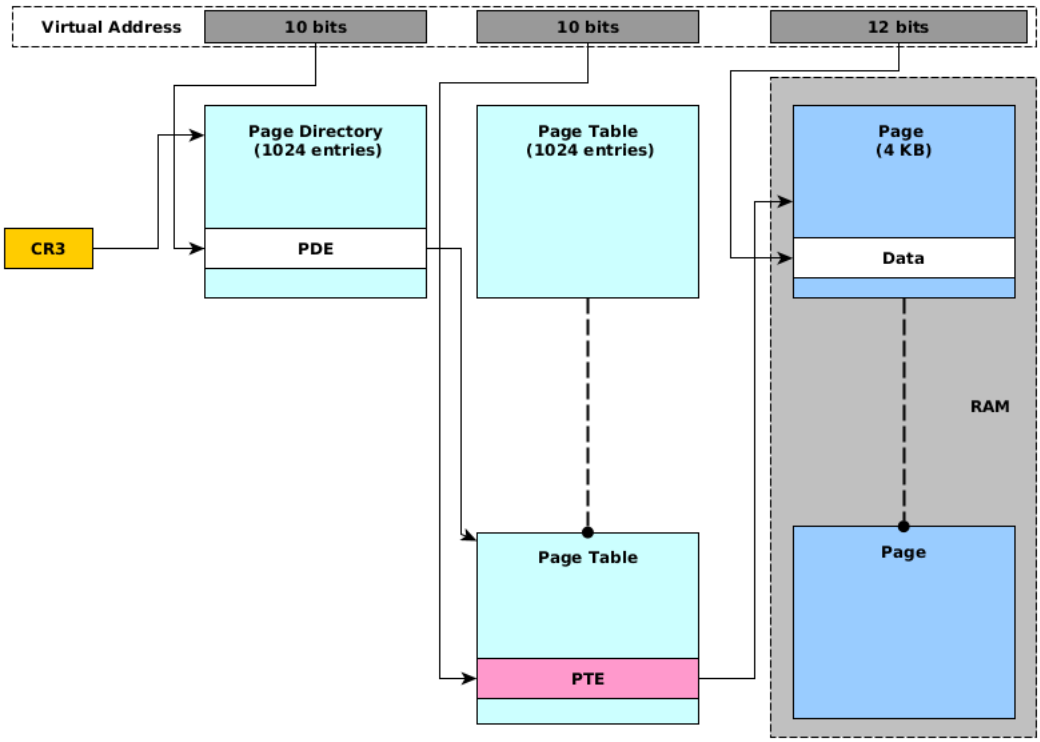

Figure 1.7: Page walking mechanism in x86_32 architecture

1.2.3 Address translation in virtualized systems

In a virtualized system, another level of memory virtualization is required. In other words, one has to virtualize the MMU to support the guest OS. The guest OS continues to control the mapping of guest virtual addresses to guest physical addresses, but the guest OS cannot have

direct access to physical/machine memory. The VMM is responsible for mapping guest physical memory to machine memory. Figure 1.8 illustrates the memory architecture in a virtualized system.

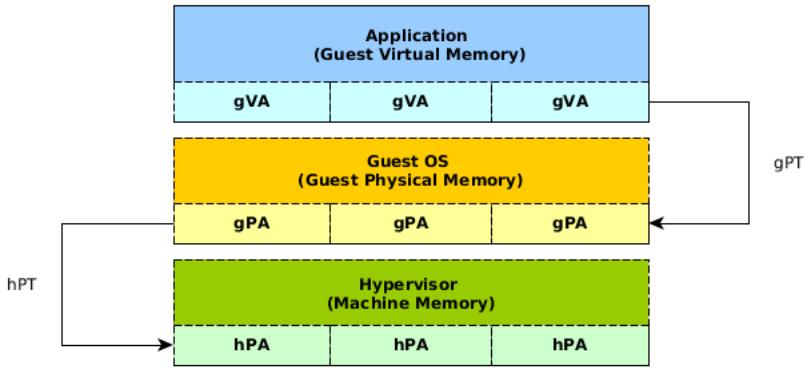

Figure 1.8: Memory architecture in a virtualized system

To fully virtualize memory therefore, two levels of address translation are required: **gVA→gPA** : guest virtual address to guest physical address translation via a per-process guest page table (gPT) and **gPA→hPA** : guest physical address to host physical address translation via a per-VM host page table (hPT), as illustrated in figure 1.9.

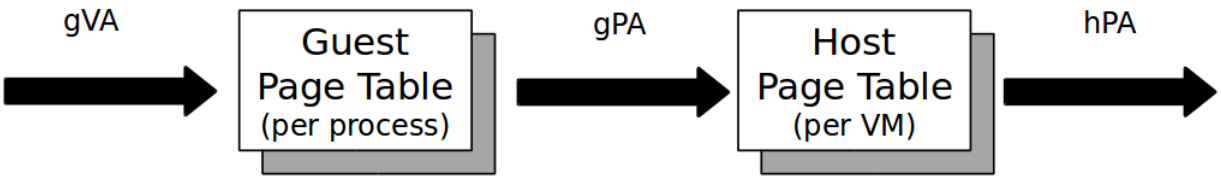

Figure 1.9: Address translation in virtualized systems

Page walks at both levels are done exactly the same way as previously illustrated.

1.2.3.1 Dimensionality of Page Walks

The dimension of a page walk corresponds to the size of the hierarchical page table depth [12]. A native page walk is usually referred to as a **1D page walk** since the hierachical page table depth is 1. That is, we traverse only one page table during a page walk. With a nested page table for example, we traverse 2 page tables simultaneously. Thus, it is referred to as a **2D page walk**. Ideally, if an application can use physical addresses directly without any page tables, this would termed a **0D page walk** [12].

With an increase in the dimensionality of page walks, the overheads associated with address translation increase. This is caused by longer latency page table walks as a result of the multi-

dimensional page tables. This increase impacts the performance of applications running in the system.

1.3 Xen hypervisor

As we mentioned above, **Xen** is a **Type-1 or baremetal hypervisor**. It is the only Type-1 hypervisor that is available as open source and as such is used as the basis for a number of different commercial and open source applications, such as: server virtualization, Infrastructure as a Service (IaaS), desktop virtualization, security applications, embedded and hardware appliances. We would use Xen as our hypervisor during the entire course of our work. The purpose of this section is to give a brief understanding of the internals of the Xen virtualization system.

1.3.1 The Xen Architecture

The Xen virtual environment consists of several components that work together in harmony to deliver a virtualization environment any customer is looking to deploy. The 3 main components in Xen architecture are: the **Xen Hypervisor, the Domain 0 Guest, and the Domain U Guests**, as illustrated in the figure below.

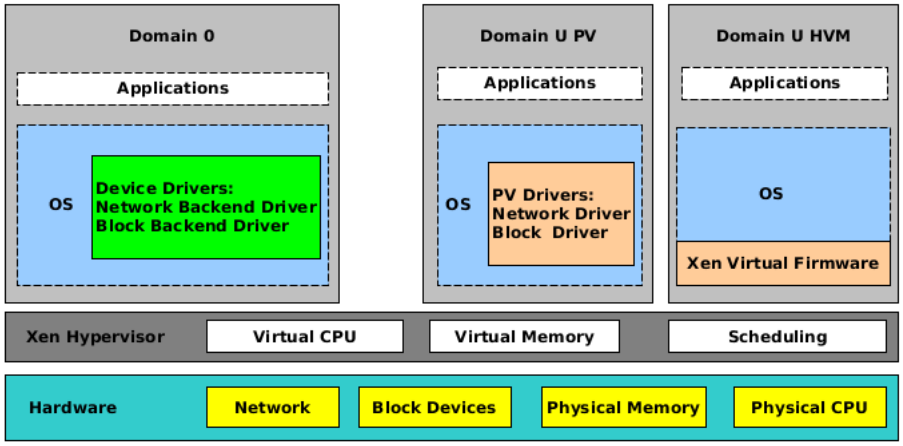

Figure 1.10: Xen virtualization system

Xen Hypervisor. The Xen hypervisor is the basic abstraction layer of software that sits directly on the hardware below any operating system. It is responsible for CPU scheduling and memory partitioning for the various virtual machines running on the hardware device. The hypervisor not only abstracts the hardware for the virtual machines but also controls the execution of virtual machines as they share the commong processing environment. It has no knowledge of networking, external storage devices, video, or any other common I/O functions found on a computing system [11].

Domain 0 Guest. The Domain 0 guest is a modified Linux kernel, and is a unique virtual machine running on the Xen hypervisor that has special rights to access physical I/O resources as well as interact with other virtual machines (Domain U: PV and HVM Guests) running on the system. All Xen virtualization environments require Domain 0 to be running before any other VMs can be started.

Two drivers are included in Domain 0 to support network and local disk requests from Domain U Guests: the **Network Backend Driver** and the **Block Backend Driver**. The Network Backend Driver communicates directly with the local networking hardware to process all VM requests coming from the Domain U guests. The Block Backend Driver communicates with the local storage disk to read and write data from the drive based upon Domain U requests.

Domain U Guest. The "U" here stands for "**unprivileged**". Domain U Guests are therefore unprivileged domains and have no direct access to physical hardware on the machine as the Domain 0 Guest does. All paravirtualized machines running on a Xen hypervisor are referred to as **Domain U PV Guests** and are modified Linux operating systems. All fully virtualized machines running on a Xen hypervisor are referred to as as **Domain U HVM Guests** and run standard Linux or Windows, or any other unchanged OS.

The Domain U PV Guest is aware that it does not have direct access to the hardware and recognizes that other VMs are running on the same machine. The Domain U HVM Guest is not aware that it is sharing processing time on the hardware and that other VMs are present. A Domain U PV Guest contains two drivers for network and disk access: **PV Network Driver** and **PV Block Driver.** The A Domain U HVM Guest does not have the PV drivers located within the virtual machine; instead a special daemon is started for each HVM Guest in Domain 0, Qemu-dm. Qemu-dm supports the Domain U HVM Guest for networking and disk access requests. The Domain U HVM Guest must initialize as it would on a typical machine so software is added to the Domain U HVM Guest, Xen virtual firmware, to simulate the BIOS an operating system would expect on startup.

This chapter has given us enough background on the topic we are working on and the different terms to be familiar with in order to understand our work as a whole. In the next chapter, we would introduce our problem statement and discuss the different state of the art techniques of memory virtualization.

CHAPTER 2

Problem statement and state of the art

In this chapter we give a concise description of the issues surrounding address translation, after which we discuss the different state-of-the-art techniques of address translation in virtualized systems. The outline of this chapter is as follows:

Contents

2.1 Problem Statement **16**

2.1.1 Address translation overheads in x86_64 16

2.2 State of the art address translation techniques in virtualized systems **18**

2.2.1 Hardware-based Nested Paging 18

2.2.2 Hypervisor-enabled Shadow paging 19

2.2.3 Other Related Work 21

2.1 Problem Statement

Most computer architectures today (e.g. Intel, ARM) employ radix trees to translate virtual addresses (VA) to physical addresses (PA) and to enforce the memory access protection dictated by the operating system [3]. The key benefit of using a radix tree is that it can serve both sparse and dense virtual address spaces, but this generality comes at a cost. Poor **memory access locality**[1] workloads, which commonly characterize modern virtual machines and large-memory databases, can benefit from more efficient mapping methods.

Let us begin by examining common radix tree-based virtual memory mechanisms, using the x86_64 architecture [2] implementation as a case study, and discuss the performance implications of this design choice.

2.1.1 Address translation overheads in x86_64

2.1.1.1 Native address translation

Processors based on the x86_64 architecture use a radix tree of depth 4 for virtual to physical address translation in native systems [3]. We denote each layer in the tree as L_i, where $1 \leq i \leq 4$. Each node in the tree corresponds to a single physical memory frame which contains the physical frame numbers (memory addresses) of the child nodes. Leaf nodes hold the target frame number to which the source virtual address is mapped.

The address translation process is iterative and traverses a path from the root node to a leaf node. In each step, subsequent bit sets from the source virtual address are used to index the pages/nodes in the subsequent levels along the search path. Figure 2.1 gives a simple illustration of this process.

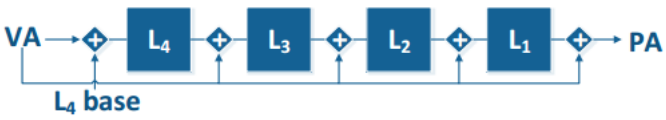

Figure 2.1: x86_64 native $VA \rightarrow PA$ translation
Source: [3]

Discussion. The performance penalty of a valid page walk in x86_64 native systems is **4 memory accesses**; Each memory access is done at each level of the page table structure/radix tree. This excessive overhead is mitigated using aggressive translation caches, which accelerate translation when workloads exhibit locality in virtual address space.

[1] A term for the phenomenon in which the same or related storage locations are frequently accessed.
[2] 64 bit x86 architecture.

2.1.1.2 Address translation in virtualized systems

Virtual machines require additional complexity when dealing with virtual address translation. Each process in a VM needs its own isolated virtual address space; Consequently, the system must provide translation from guest virtual addresses (gVA) to physical addresses on the host machine/host physical addresses (hPA). This translation is performed using a **nested page table**, which consists of a guest page table (gPT) in the VM and host page table (hPT) in the hypervisor. Each node in the guest radix tree points to a node in the next level using a guest physical frame number (which is still a host virtual address), and thus each transition between subsequent levels in the guest radix tree incures a full $gPA \rightarrow hPA$ using the hPT. This is referred to as a **two-dimensional (2D) page walk**. Figure 2.1 illustrates a 2D page walk in x86_64 systems.

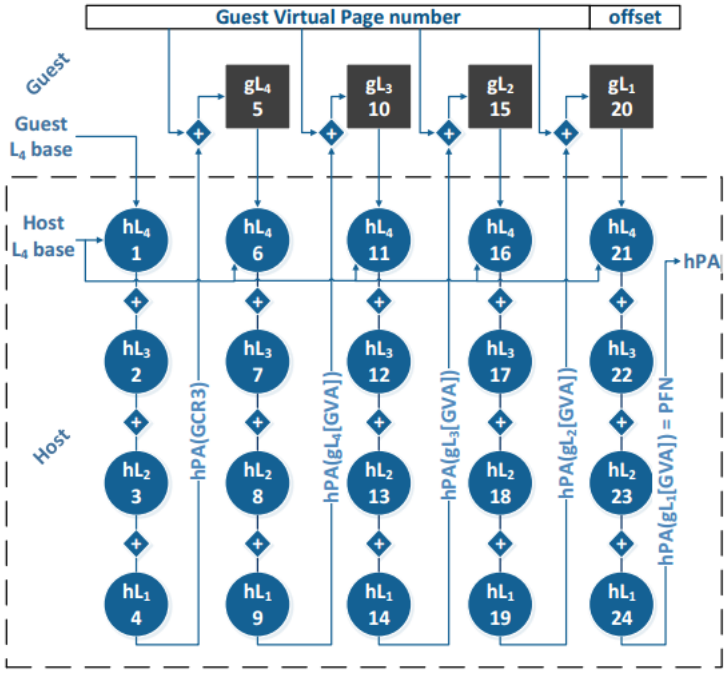

Figure 2.2: x86_64 native $gVA \rightarrow hPA$ translation
Source: [3]

Discussion. Nested $gVA \rightarrow hPA$ translation dramatically increases the number of memory accesses to a total of **24 memory accesses**. This hugely affects the run time of VM workloads. Figures 2.3 and 4.3 show the fraction of the run time devoted to page table walks when running a set of benchmarks on a native system and virtualized system respectively, using 4KB pages in both cases.

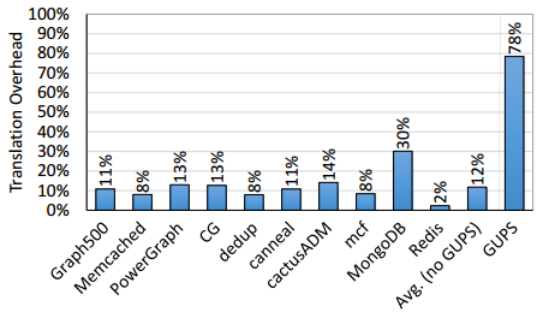

Figure 2.3: Native applications, 4KB pages
Source: [3]

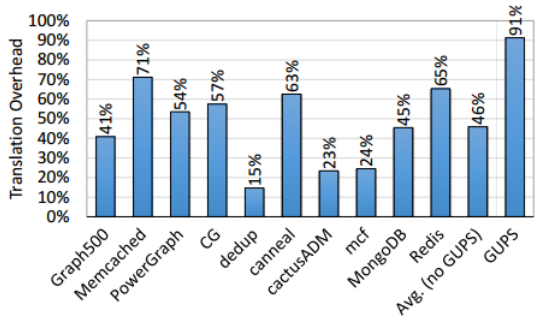

Figure 2.4: Applications running in VM, g4KB/h4KB pages
Source: [3]

The above figures clearly illustrate the dramatic increase in translation overheads in VM workloads when compared to their native counterparts. This huge performance penalty motivates us to rethink how we perform address translation in virtualized systems, which leaves us with an interesting challenge:

Questions

- How can we perform address translation in virtualized systems and still preserve base native performance ?

2.2 State of the art address translation techniques in virtualized systems

There are two state-of-the-art techniques of memory virtualization in virtualized system: **Nested Paging** and **Shadow Paging**.

2.2.1 Hardware-based Nested Paging

Nested paging is a widely used hardware technique to virtualize memory [6]. Here, the processor has two hardware page table pointers to perform a complete translation: one points to the guest page table **(gptr)** and the other points to the host page table **(hptr)**. The guest page table holds gVA to gPA translations and the host page table holds gPA to hPA translations.

In the best case, the virtualized address translation hits the TLB to directly translate from gVA to hPA with no overheads. In the worst case, a TLB miss needs to perform a 2D page walk that multiplies overheads vis-à-vis native, because accesses to the guest page table also requires translation by the host page table. We perform 4 memory references to translate the **gptr** (since each gPA requires access to the host page table), 4 memory references for each of the 4 levels of

the host page table, and 4 memory references for the guest page table itself. This gives us a total of 24 memory accesses.

Various caching techniques in today's commodity processors, like caching page table entries in data caches [14], MMU caches [5, 8] and caching intermediate translations [6, 7] can prevent some of the memory references resulting from a TLB miss. Even though a virtualized TLB miss is longer than a native TLB miss, nested paging allows fast and direct updates to both of the page tables without any VMM intervention.

2.2.1.1 Advantages of Nested Paging

- Allows fast and direct updates to the guest and host page tables without any traps in the hypervisor.

2.2.1.2 Disadvantages of Nested Paging

- On a TLB miss, the MMU performs a 2D page walk that requires up to 24 memory references; The overhead incurred is pretty huge.

2.2.2 Hypervisor-enabled Shadow paging

Shadow paging is a less used software technique to virtualize memory [12]. With shadow paging, the VMM creates a **shadow page table** (on demand) by merging the guest and host page tables. The shadow page table holds complete translations from gVA to hPA, which means using shadow page tables can potentially eliminate page walks for nested page tables. However, maintaining shadow page tables incurs significant performance and memory overheads, which often exceeds the benefits of using shadow page tables. Figure 2.5 illustrates the idea behind shadow paging.

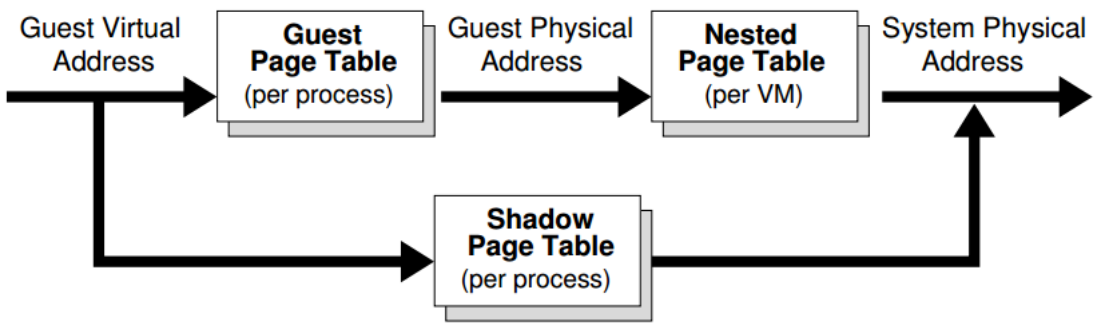

Figure 2.5: Shadow paging

Shadow page tables employ the same multi-level page walks as guest page tables, and thus the memory references required for a shadow page table walk are the same as for a base native page walk, i.e.**4 memory references** for a complete gVA to hPA translation.

The primary performance overhead of shadow paging is the cost of hypervisor interventions. Exiting from a **guest VM context** to the **hypervisor context (VM exit)** incurs a significant performance overhead in virtualized systems. To maintain consistent shadow pages, there have been two types of mechanisms to detect guest page table changes. Firstly, the hypervisor restricts the memory area of guest page tables to be **read-only**. Any attempt by a guest OS to update guest page tables will be caught by the hypervisor. The alternative way, called **virtual TLB**, is to let a guest OS to modify a guest page table, and detect the change later if the corresponding shadow page table entry does not exist during an actual memory access to the page. In this mechanism, TLB invalidation instructions also invoke the hypervisor, as the guest OS executes them for TLB consistency, when it removes page table entries from the guest page tables.

If a **page fault** occurs, the hypervisor must intervene and may have to traverse both the guest and shadow page tables, since it does not know whether the page fault must be handled by the guest OS or the hypervisor [2]. Furthermore during a process **context switch**[3] within a VM, the hypervisor should replace the guest page table pointer with the corresponding shadow page table pointer, so that the hardware page walker uses the shadow page table instead of the guest page table.

2.2.2.1 Advantages of Shadow Paging

- Only 4 memory references (same as base native) required for a complete page walk on a TLB miss.
- No need for any extra hardware support for page walks.

2.2.2.2 Disadvantages of Shadow Paging

- Every page table update requires a costly trap in the hypervisor to update shadow page table entries.
- There are costly traps in the hypervisor during context switches in order to keep the shadow page table consistent [12].

Figure 2.6 represents pseudocode for page walk state machines for native and existing virtualized address translation techniques.

[3] Switching of the CPU from one process to another.

```
host_walk (VA, hptr)
PA = hptr;
for (i=0; i≤MAX_LEVELS; i++) do
  | PA = host_PT_access (PA + index(VA,i)) ;
end
return PA;
```

(a) Base native page walk state machine

```
nested_walk (gVA, gptr, hptr)
//page fault in hPT will cause a VM exit
hPA = host_walk (gptr, hptr) ;
for (i=0; i≤MAX_LEVELS; i++) do
  | hPA = nested_PT_access (hPA + index(gVA,i), hptr) ;
end
return hPA;
```

(b) Nested page walk state machine

```
shadow_walk (gVA, sptr)
return host_walk (gVA, sptr) ;
```

(c) Shadow page walk state machine

```
host_PT_access (address)
PTE = *address;
PA = PTE.PA;
if PTE.valid == 0 then
  | raise host PAGE_FAULT;
return PA;
```

(d) Host page table access helper function

```
nested_PT_access (address, hptr)
PTE = *address;
gPA = PTE.PA;
if PTE.valid == 0 then
  | raise guest PAGE_FAULT;
//page fault in hPT will cause a VM exit
hPA = host_walk (gPA, hptr) ;
return hPA;
```

(e) Nested page table access helper function

Figure 2.6: Pseudocode for native, nested, and shadow page walk state machines.

2.2.2.3 Comparison of Trade-Offs

Table 2.1 summarizes the trade-offs provided by both shadow and nested paging as compared to base native.

	Base Native	**Nested Paging**	**Shadow Paging**
TLB Hit	fast	fast	fast
Max. memory access on TLB miss	4	24	4
Page table updates	fast direct	fast direct	slow mediated by VMM
Hardware support	1D page walk	2D page walk	1D page walk

Table 2.1: Trade-offs provided by shadow and nested paging as compared to base native

From the table Nested and Shadow paging both present significant overhead due to memory accesses and page table updates respectively, when compared to base native. With current hardware and software, the overheads of virtualizing memory are hard to minimize because a VM exclusively uses one technique or the other.

2.2.3 Other Related Work

The previous sections discussed state-of-the-art techniques of memory virtualization. In this section, we present other related work that comprises mostly research proposals geared towards address translation overhead mitigation in virtualized systems.

2.2.3.1 Flat Page Tables for VMs

Ahn et al.[2] proposed replacing x86_64's 4-level nested page table with a flat 1-level nested page table. This reduces the number of nested page table accesses from 4 to 1 and a 2D page walk from 24 to 8 memory references in total. While this is promising for the small virtual machines they studied, it may be less suited for big-memory workloads [12].

2.2.3.2 Selective Hardware Software Paging (SHSP)

Past work - Selective Hardware Software Paging (SHSP) - showed that a hypervisor could dynamically switch an entire guest process between nested and shadow paging to achieve the best of either techniques [28]. It monitored TLB misses and guest page faults to periodically consider switching to the best mode (less translation overhead). However, switching to shadow mode requires (re)building the entire shadow page table, which is expensive for multi-GB to TB workloads.

2.2.3.3 Multipage Mappings

Multipage Mapping approaches, such as sub-blocked TLBs [23], CoLT [18] and Clustered TLBs [22], pack multiple PTEs into a single TLB entry. These designs leverage default OS memory allocators that either assign small blocks of contiguous physical pages to contiguous virtual pages (Sub-blocked TLBs and CoLT), or map small sets of contiguous virtual pages to clustered sets of physical pages (Clustered TLB). However, they pack only a small multiple of translations per entry, which limits their potential to reduce page walks for large workloads.

In this chapter, we discussed the main issues regarding address translation in virtualized systems and the different state-of-the-art techniques of memory virtualization. In the next chapter we would discuss our contribution and all its merits.

CHAPTER 3

CONTRIBUTION

In this chapter we provide a detailed theoretical description of our solution, along with the different advantages provided by our mechanism of address translation. The outline of this chapter is as follows:

Contents

3.1 Proposed Model **24**

3.1.1 Basic Idea 24

3.1.2 Detailed description of our solution 25

3.2 Reducing Fragmentation **26**

3.2.1 Memory Compaction 26

3.2.2 Reclaiming I/O gap memory 27

3.3 Advantages of our mechanism **27**

3.3.1 Comparison of trade-offs 28

3.1 Proposed Model

In the previous chapter, we discussed state-of-the-art techniques of memory virtualization and presented a summary of trade-offs provided by these techniques as compared to base native. We saw that nested paging for example, presented significant overhead due to large numbers of memory accesses on TLB misses. To solve this problem we propose a new mechanism of address translation in virtualized systems based on **contiguous memory** virtual machines. This solution would permit us to **nullify address translation overhead vis-à-vis base native**. We take inspiration from **direct segments** [12], which was proposed as a solution to reduce TLB misses in unvirtualized (native) systems.

3.1.1 Basic Idea

The basic idea behind our address translation mechanism is to **allocate contiguous physical memory** to virtual machines and introduce a hardware feature to store the **base address** of the contiguous chunk of machine memory. That is, we try to map a VM's guest physical address space to a chunk of contiguous host physical addresses. Figure 3.1 illustrates this idea.

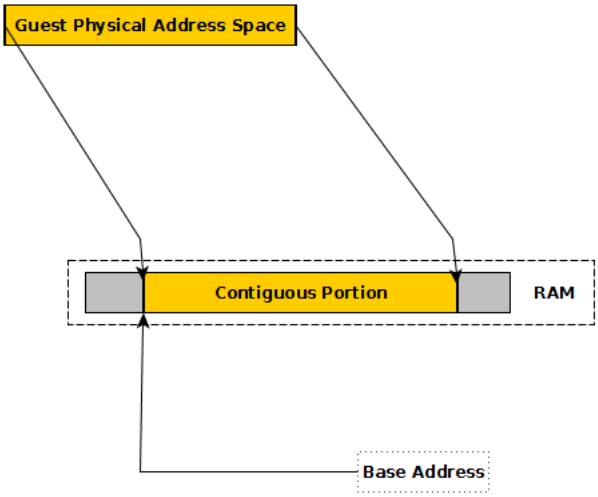

Figure 3.1: Mapping guest physical address space to a contiguous portion of RAM

With such a memory layout,a virtual address gets translated to a physical address via simple addition, as shown in the equation below:

$$\mathbf{hPA = base_address + gPA}$$

Figure 3.2 illustrates this mechanism of address translation.

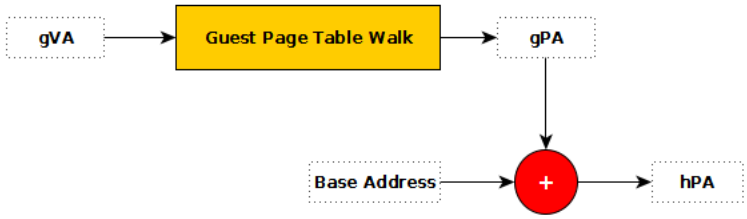

Figure 3.2: Our translation mechanism

3.1.2 Detailed description of our solution

3.1.2.1 General Architecture

In this section, we give a more detailed explanation of our address translation technique. Figure 3.3 depicts the overall flow of this mechanism.

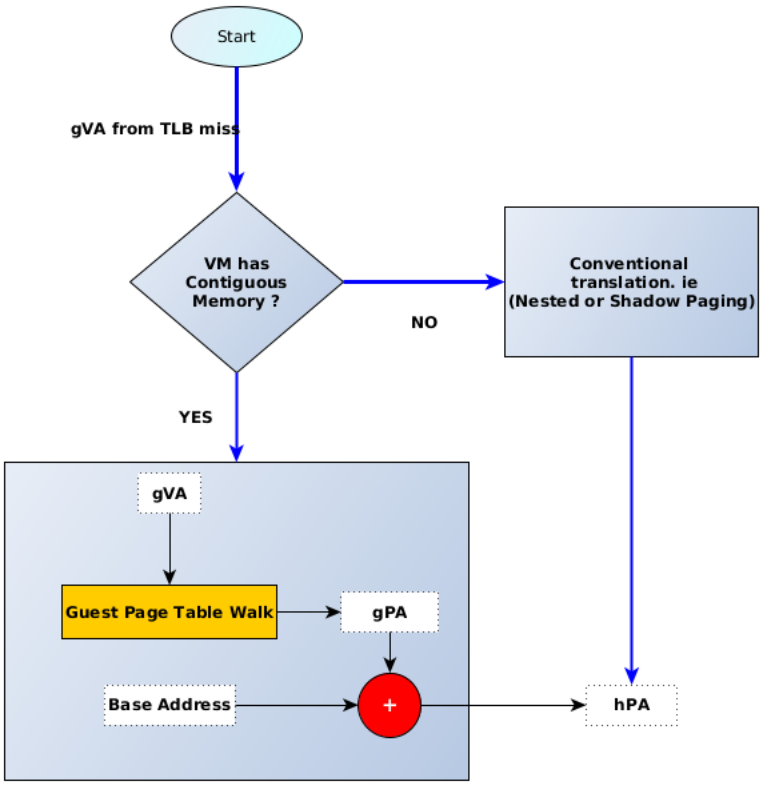

Figure 3.3: General architecture of our model

When a VM is being created, the hypervisor tries to allocate a contiguous chunk of machine memory to it. If it succeeds in doing this mapping, the hypervisor stores the **base address** of this contiguous chunk in a hardware register.

When a guest virtual address (gVA) is accessed, the TLB is checked first for the existence of a mapping. If the mapping is not found (TLB miss), the hypervisor starts by verifying if the guest physical address space is mapped to a contiguous piece of machine memory (RAM). If that is not the case, then the guest physical address space is not compatible with our address translation mechanism and the hypervisor uses conventional methods of address translation (nested or shadow paging) to perform the translation. In the ideal case, where the guest physical address space is mapped to a chunk of machine memory, the guest OS starts by performing a conventional guest page table walk (**4 memory references**) using the different bits of the gVA. The outcome of this page walk is the corresponding gPA. The VMM then reads the base address in the hardware register and adds the gPA to this base address in order to obtain the host physical address (hPA) or real address corresponding to the gVA. So we have a total of 4 memory references (same as base native) for a complete address translation.

3.1.2.2 Enforcing Memory Protection and Isolation

Our model relies on the value of the base address and the guest VM's size to ensure protection and isolation of VMs. When a VM is scheduled, the **guest Base Address (gBA)** and the **guest Memory Size (gMS)** are included within the **virtual machine control structure (VMCS)**[1]. To ensure isolation, the TLB is enhanced with a security module such that during insertion into the TLB, the security module checks if $hPA \leq gBA + gMS$. This is enough to ensure that the VM is not operating in a region of physical memory that belongs to another VM.

3.2 Reducing Fragmentation

The continuous creation and destruction of VMs with contiguous chunks of physical memory could prevent the allocation of contiguous machine memory at a certain point in time due to **fragmentation**. Moreover, the x86_64 architecture fragments the guest physical memory of VMs for **memory-mapped I/O**[2] [12]. We discuss a few ways to reduce memory fragmentation so as to increase the probability of contiguous memory allocation.

3.2.1 Memory Compaction

As physical pages are being allocated from a memory pool, the pool becomes fragmented. If some process requests a large piece of storage from the OS, the OS may not satisfy that request because there is no single contiguous chunk of memory that is large enough to satisfy the request,

[1] A data structure which contains the state/context of a VM.

[2] A scheme where an I/O device is mapped into the address space of the system.

even though the total amount of free/unused storage in the pool is large enough. **Memory compaction** is simply the rearrangement of free and used pages so that the OS can satisfy the request.

Compaction is supported in many operating systems including Linux [9]. Once the compaction daemon[3] provides enough contiguous physical memory the VMM can allocate this to a VM upon creation.

3.2.2 Reclaiming I/O gap memory

A second source of fragmentation in VM guest physical memory is architectural. The x86_64 architecture has an I/O gap, which is a 1GB region at the high-end of the 32-bit (4GB) guest physical address space reserved for memory-mapped I/O [14]. By default, this gap splits the addresses backed by guest physical memory to about 3GB before the I/O gap, which prevents the hypervisor from mapping all guest physical memory to a single contiguous chunk of machine memory all at once.

We prevent this by modifying the hypervisor heap allocator in such a way that all pages are allocated in an organized contiguous fashion.

3.3 Advantages of our mechanism

Our mechanism of address translation has the following advantages:

Less memory references. As mentioned earlier, a full address translation requires a maximum of 4 references. This significantly reduces translation overhead which is almost equal to that of base native.

Less VM exits. Our solution greatly reduces overhead as a result of VM exits; With our solution, the gPT is not read-only and as such VMs do not need to exit each time the gPT needs to be modified. Furthermore, most hypercalls that were necessary for the operation of a non-contiguous memory VM could be eliminated, since the guest OS now has direct access to machine memory.

Security and Isolation. For each hPA, our security module checks if $hPA \leq gBA + gMS$. This is a very simple and straightforward calculation which ensures security and isolation at the hardware level.

[3] A process that runs in the background on a multi-tasking OS.

3.3.1 Comparison of trade-offs

Table 3.1 shows us a theoretical comparison of trade-offs provided by our translation model as compared to shadow and nested paging, and base native.

	Base Native	**Nested Paging**	**Shadow Paging**	**Our Solution**
TLB Hit	fast	fast	fast	fast
Max. memory access on TLB miss	4	24	4	4
Page table updates	fast direct	fast direct	slow mediated by VMM	fast direct
Hardware support	1D page walk	2D page walk	1D page walk	1D page walk

Table 3.1: Trade-offs provided by our solution

We notice that trade-offs provided using our solution are identical to base native. Our solution is clearly more efficient when compared to shadow or nested paging.

We have described the inner workings of our address translation model; In the following chapter, we would discuss our proof of concept and the different implementation details of our mechanism, as well as the various simulation results obtained.

CHAPTER 4

Implementation and Evaluation

Here we discuss the implementation details of our model as regards simulations and hypervisor implementation. We then present our simulation results along with a detailed analysis of these results. The outline of this chapter is as follows:

Contents

4.1 Implementation **30**
4.1.1 Simulation 30
4.1.2 Hypervisor modification 32
4.1.3 Guest OS 35
4.1.4 Development tools 36
4.2 Evaluation **36**
4.2.1 Evaluation Methodology 36
4.2.2 Experimental Setup 39
4.2.3 Simulation Results 39
4.2.4 Analysis of Results 41

4.1 Implementation

4.1.1 Simulation

4.1.1.1 Proof of Concept

We can recall from the previous chapter that our address translation mechanism applies on VMs with **contiguous host physical memory**. That said, a study or **proof of concept** needs to be done so as to give us an idea of the percentage of VMs in real data centers with contiguous machine memory. In more formal terms, a proof of concept is simply a demonstration that proves the potential feasibility of an idea.

For our feasibility study, we focused on aspects of real cloud workloads that have an impact on resource management. Thus, all of our datasets contain information about VM size (CPU cores and Memory size) and VM creation and deletion timestamps. We used **VM traces** from the following data centers:

- Microsoft Azure
- CERIT Scientific Cloud
- Bitbrains

We then developed a **simulator** to replay these traces using different **VM placement algorithms**. **VM placement** is the process of selecting the most suitable server in a large cloud data center to deploy newly-created VMs [13]. Our simulation results provide an estimate of the percentage of VMs with contiguous memory during their lifetime.

4.1.1.2 Simulation Setup

To begin, let us introduce the simulation setup we used to estimate the percentage of contiguous memory VMs in datasets. For each dataset, we consider the resource requirement of each VM in terms of CPU cores and memory. We note that upon creation, each VM instance requires various resources such as number of virtual processors, memory and storage with different capacities. VM objects are created and destroyed according to their creation and destruction timestamps respectively. We take into consideration server generations and their resource capabilities for each dataset so as to provide more realistic results. We further assume that the resource capabilities of a datacenter are capable to host all the VMs in a given dataset. Our simulation algorithm simulates 2 major VM placement algorithms:

Contiguity Aware Placement: This algorithm prioritizes servers that can provide a VM with contiguous memory. That is, enough contiguous physical memory pages corresponding to the size of the VM's memory request. If no running server can satisfy the request, a new server is turned on and used. In case all servers are running and none can provide contiguous memory,

the first server satisfying the resource request is chosen. The following pseudocode illustrates the contiguous aware placement algorithm.

Algorithm 1 Contiguity Aware VM Placement algorithm

```
function CHOOSECONTIGUOUSSERVER(vm,serverList)
    for i ← 0 to serverList.size do
        tempServer = serverList.get (i);
        if tempServer.hasContiguousMemory (VM) == true then
            return tempServer
        end if
    end for
    if tempServer == NULL then
        tempServer = possibleServers.get(0);
    end if
    return tempServer
end function
```

Traditional Placement (Min/Max): This algorithm simply takes into consideration resource availability in physical servers. The **Min** algorithm chooses the physical server with the least available resources among all the possible running servers that can host the VM instance. If no running server has enough available resources, a new server is turned on and used. The **Max** algorithm chooses the physical server with the most available resources among all the possible servers that can host the VM instance. The following pseudocode illustrates the Max and Min traditional placement algorithms.

Algorithm 2 Max Placement algorithm

```
function CHOOSEMAXSERVER(vm,serverList)
    availResources = vm.requiredResources;
    for i ← 0 to serverList.size do
        tempServer = serverList.get (i);
        if tempServer.availResources >= availResources then
            maxServer = tempServer;
        end if
    end for
    if maxServer == NULL then
        maxServer = new Server();
    end if
    return maxServer
end function
```

Algorithm 3 Min Placement algorithm

```
function CHOOSEMINSERVER(vm,serverList)
    availResources = vm.requiredResources;
    for i ← 0 to serverList.size do
        tempServer = serverList.get (i);
        if tempServer.availResources >= availResources then
            possibleServers.add (tempServer);
        end if
    end for
    min = possibleServers.get(0).availResources;
    for i ← 0 to possibleServers.size do
        tempServer = possibleServers.get (i);
        if tempServer.availResources <= min then
            minServer = tempServer;
        end if
    end for
    if minServer == NULL then
        minServer = new Server();
    end if
    return minServer
end function
```

4.1.2 Hypervisor modification

As we mentioned earlier, we used the **Xen hypervisor** to implement our address translation mechanism. This entailed an in depth modification of Xen code to take into account our new design. Xen is written in **C, Assembly Language** and **Python**, and its source code comprises about 5000 files containing more than a million lines of code [24]. The latest release is Xen is **4.10** but we used version **4.7** for stability issues.

4.1.2.1 Contiguous Memory Allocation

This was the very first part of our hypervisor implementation and the importance of this should be obvious: our solution is based on contiguous memory VMs. In order to proceed, we must first understand what happens when a Xen VM is created.

Xen uses a **configuration file** to create a VM. The configuration file contains information such as the amount of memory (in MB) to allocate to a VM, number of virtual CPUs, network related information etc. For the sake of simplicity, we would limit ourselves to memory related information. Xen uses the memory information in the configuration file to estimate the number of physical pages to be allocated to the VM. Recall, in x86 systems, a physical page is **4KB** in size. These pages are then allocated and mapped to the guest physical address space of the VM.

How Xen allocates physical memory pages: In Xen, the **Xen Heap** is in charge of allocating free pages to VMs on demand. The Xen Heap is a specialized data structure which contains all the addresses of free memory segments and pages in machine memory. By default, Xen allocates pages in batches of **4KB**, **2MB**, or **1GB**. For efficiency reasons, the Xen Heap does not allocate pages 4KB at a time. To allocate a batch of pages, Xen must calculate the **order** corresponding to the number of pages in the batch and pass this order to the Xen heap allocator. The relationship between the order and the number of pages is given below:

$$2^{order} = \textbf{Number of 4KB pages}$$

This means a group of 4 contiguous pages has an order of 2, a group of 512 contiguous pages (2MB) has an order of 9 and a group 262,144 4KB contiguous pages (1GB) has an order of 18. A single 4KB page would have an order of 0. To allocate, for example, a 1GB chunk of memory pages, we need to pass the order 18 to the Xen heap allocator. In Xen, contiguous chunks of physical pages are referred to as **extents.** The maximum order of an extent Xen can handle is 20, which corresponds to an extent of 4GB. To understand how the Xen heap finds extents of different orders, it is important for us to master the structure of the Xen heap.

Structure of the Xen heap: The Xen heap is tree-like in structure and the heap itself could be seen as the root of the tree. The heap consists of nodes and each node consists of zones. Each zone comprises a number of lists which contain the addresses of extents of a specific order. Each list is indexed by the order of the extents it contains. A list L_i for example, would contain the addresses of free extents of order **i** within that zone. Figure 4.1 illustrates this.

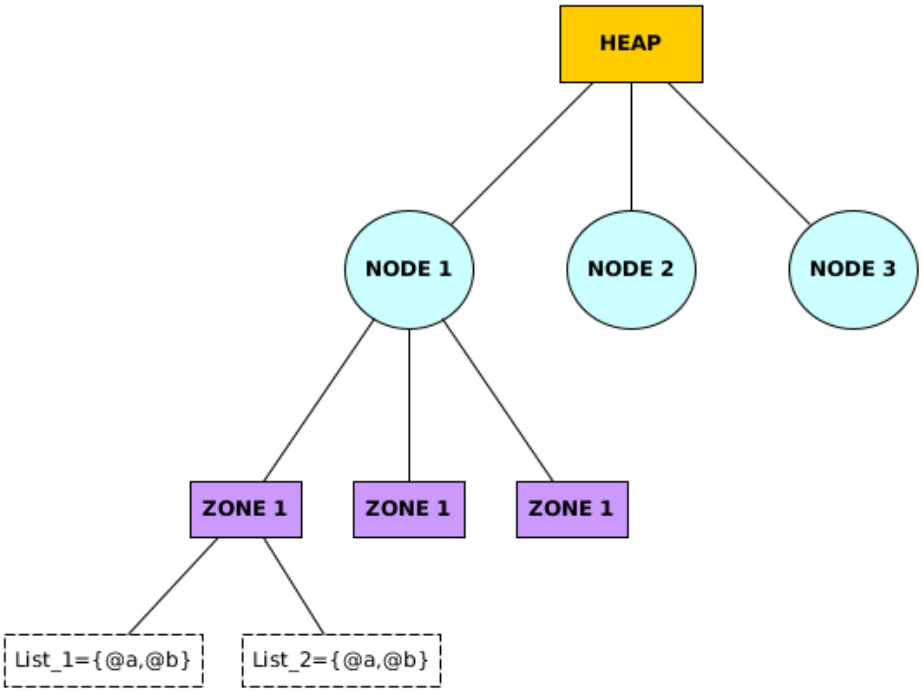

Figure 4.1: The Xen Heap Structure

As we can see, the Xen heap has a very elegant design but we have a little problem at the level of contiguity: the Xen heap does not necessarily allocate extents in a contiguous fashion. To solve this problem, we had to modify the heap allocator in such a way that it allocates all extents for Domain U guests in a contiguous fashion. NB: To know if a group of pages are contiguous, we calculate the **machine frame number (MFN)**[1] of each page and check if the MFNs of all the pages are consecutive. The following pseudocode gives a brief summary of the core modifications we made in the Xen heap allocator so as to achieve our goal.

[1] A number that identifies a page/frame in RAM memory.

Algorithm 4 Xen Heap Allocator

```
function ALLOC_HEAP_PAGE(order,node)
    zone = zone_hi;
    while zone > zone_lo do
        for i ← order to MAX_ORDER do
            if (!page_list_empty(heap(node,zone,i))) then
                for all (page in heap(node,zone,i)) do
                    if page.mfn == next_mfn then
                        next_mfn+=page.size;
                        goto found;
                    end if
                end for
            end if
        end for
        zone - -
    end while
    found: ...
end function
```

The Xen heap allocator is of course more complex than what is illustrated in the above code. Nevertheless, the above algorithm gives us an idea of how we force the Xen heap allocator to allocate pages in a contiguous fashion. In the case where it is impossible to allocate contiguous pages, the Xen heap allocator uses its default algorithm, and our translation mechanism cannot be applied.

4.1.2.2 Address Translation

Domain U PV guests in Xen use **direct paging** for address translation [4]. Here the guest OS uses a **physical to machine (P2M)** mapping to map guest physical addresses to machine addresses. P2M mappings map guest physical addresses directly to machine addresses and as such the guest OS uses machine addresses directly when it writes its page tables. Our contiguous memory allocations make its very easy to build the P2M table in Domain U PV guests as all pages are contiguous. Furthermore, it becomes possible to eliminate unnecessary hypercalls in PV guests since they somehow have access to a specified range of contiguous machine memory.

Applying our solution to Domain U HVM guests is quite complex because we need additional hardware (which is absent in the default architecture). As such, we can only evaluate our mechanism through simulation in the case of Xen HVMs.

4.1.3 Guest OS

For research purposes, it is important to choose the right OS for our guest operating systems. We chose **Linux** for the following reasons:

- it is open source and thus we can modify its source code,
- it has a large community of developers who are quite active and ready to help on pertinent issues,
- it integrates the paravirtualization model offered by Xen.

The Linux kernel is written in **C** and **Assembly Language**. The source code of the Linux kernel comprises about 38000 files containing about 15 million lines of code. The latest release of the Linux kernel at the time of this writing is **14.17.13**. We used kernel version **4.13.0** for stability and compatibility reasons.

4.1.4 Development tools

4.1.4.1 Compiler

A suitable compiler is needed to compile Xen and the Linux kernel. We used the **GNU Compiler Collection (GCC)** in our case. GCC is free software and enables compilation of of many high level languages such as: C, C++, Objective-C, Java, Ada, and Fortran. We used version **5.4.0** for our work.

4.1.4.2 Integrated Development Environment (IDE)

To work with the Linux kernel and Xen source code, a good IDE is of great importance. This is because Xen and the Linux kernel are pretty large and navigating through their files and folders could be very complex. We used 3 main editing tools for our work: **Visual Studio Code IDE, Vim editor,** and the **Cscope navigation tool**. Cscope is a tool used for navigating huge C projects. We used version 15.8 of Cscope, version 7.4 of Vim, and version 1.23.1 of Visual Studio Code.

4.2 Evaluation

In this section, we would evaluate our idea as far as simulations are concerned. We would begin by explaining our evaluation methodology after which we would present the simulation results obtained.

4.2.1 Evaluation Methodology

We carried out simulations on datasets from Microsoft Azure, CERIT Scientific Cloud, and Bitbrains. Each dataset comprises VM traces (.csv files) which contain information about the different VMs executed over a specific period of time in a data center. For each VM trace, the simulator uses one of the different placement algorithms described previously to choose a physical server

object on which to host the VM object for its entire lifetime, which is determined by its creation and deletion timestamps. The simulator notes if the VM was allocated contiguous memory or not during its lifetime and at the end of the simulation, it calculates the percentage of VMs which were allocated contiguous memory during their lifetimes and the number of VMs and servers used for the simulation. The following section gives a brief description of the datasets we used for simulation.

4.2.1.1 Datasets

As we mentioned above, our datasets come from 3 data centers: Microsoft Azure, CERIT Scientific Cloud, and Bitbrains. The characteristics of these datasets are as follows:

Microsoft Azure dataset: This dataset contains information about VMs running on Azure from November 16, 2016 to February 16, 2017. The trace information includes: identification numbers for each VM and the deployment and subscription to which it belongs; the VM role name; the VM size in terms of its maximum core, memory and disk allocations; and the minimum, average and maximum VM resource utilizations (reported every 5 minutes). Azure hosts both first-party and third-party workloads. First party workloads comprise internal VMs (research/development, infrastructure management etc) and third-party workloads comprise VMs created by external customers [10]. Table 4.1 shows us the physical server composition we used in our simulation.

Server Generation	**%**
Generation 3	20
HPC	20
Generation 4	20
Generation 5	20
Generation 6	10
Godzilla	10

Table 4.1: Physical server composition used for simulation

The following figures present an illustration of the virtual resource usage in the Microsoft Azure dataset.

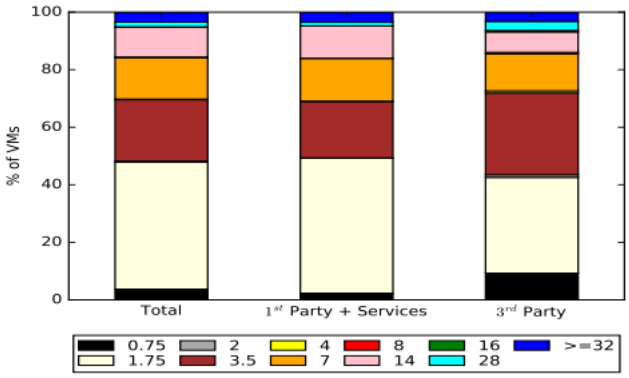

Figure 4.2: Amount of memory per VM in GB

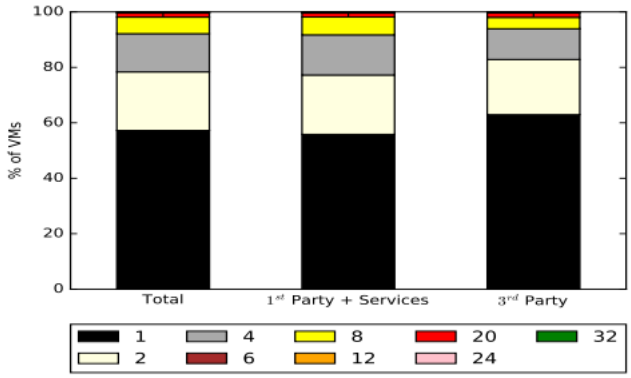

Figure 4.3: Number of virtual CPU cores per VM

CERIT Scientific Cloud dataset: This dataset represents the workload from 11,382 running VMs over a period of 1 year from CERIT Scientific Cloud [15]. The physical infrastructure of CERIT-SC's virtualized environment consists of six major clusters[2] that vary heavily by means of their size and per-node parameters. Table 4.2 represents the cluster names and charateristics:

Name	RAM(GB)	CPU(cores)
zapat	128	16
zebra	256	40
zigur	128	8
zegox	90	12
zefron	1000	40
zorg	512	40

Table 4.2: CERIT-SC cluster characteristics

Bitbrains dataset: This dataset comprises two groups of VM traces: The first group, fastStorage, consists of 1250 VMs connected to fast storage area network (SAN) devices and the second group, Rnd, consists of 500 VMs that are either connected to the fast SAN devices or the much slower Network Attached Storage (NAS) devices. The fastStorage trace includes a higher fraction of application servers while the Rnd trace includes mostly management machines which require lower storage as compared to the fastStorage VMs. VMs in the Bitbrains dataset use from 1 up to 32 CPU cores, but over 85% of the VMs use at most 4 cores. On average, VMs in the Rnd dataset use slightly fewer cores. Most VMs in this dataset demand a maximum of 8GB of RAM but the VMs in the Rnd dataset demand slightly less memory than those in the fastStorage dataset, which request memory between 1 and 512GB [19].

[2] Groups of connected physical servers.

For simulation, we used the same server composition as that used for the Microsoft Azure dataset.

4.2.2 Experimental Setup

The simulations were carried out on a physical computer with the following characteristics:

- 8 processors Intel (R) Core (TM), 2.60GHz
- 8GB RAM memory
- Ubuntu 16.04 LTS Linux Kernel 4.15

4.2.3 Simulation Results

In this section, we present the simulation results we obtained after replaying traces from the different datasets. The simulation results represent the percentage of VMs with contiguous memory using the VM placement algorithms we discussed above. NB: For each plot, the vertical axis is shifted so as to magnify the difference in percentages for the different placement algorithms.

Microsoft Azure: Figure 4.4 shows the results obtained using the Microsoft Azure dataset. We have very high numbers of VMs with contiguous memory using the different placement

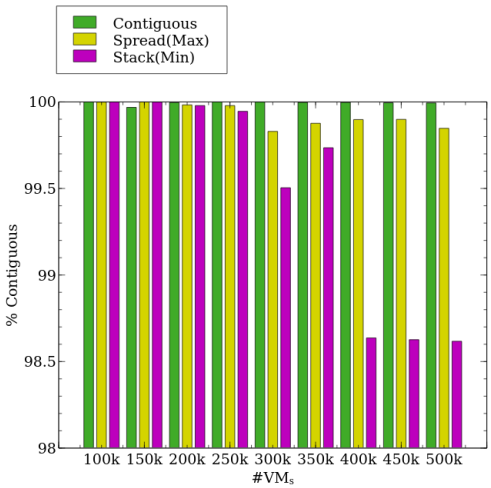

Figure 4.4: Percentage of contiguous-memory VMs in the Microsoft Azure dataset

algorithms. This is because the VM sizes are very small compared to server sizes. Nevertheless,

this shows that a pretty large percentage of VMs in this datacenter could be given contiguous memory.

CERIT-SC: Figure 4.6 shows the results obtained using the CERIT-SC dataset.

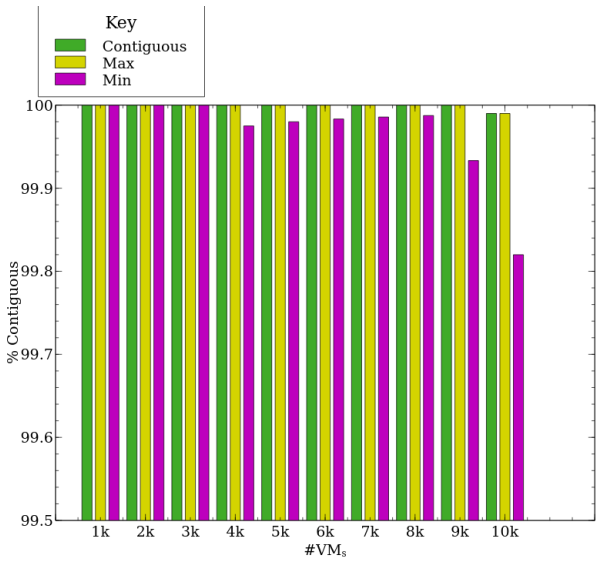

Figure 4.5: Percentage of contiguous-memory VMs in the CERIT-SC dataset

Bitbrains: Figure 4.6 shows the results obtained using the Bitbrains dataset.

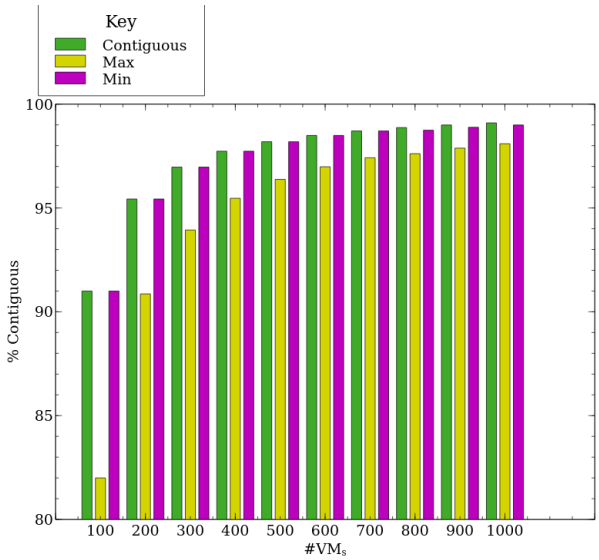

Figure 4.6: Percentage of contiguous-memory VMs in the Bitbrains dataset (fastStorage)

We still have a high percentage of contiguous-memory VMs here for each placement algorithm.

However, this percentage is lower in general when compared to the other datasets. This is because the VMs in the Bitbrains dataset have quite high resource demands compared to VMs in the other datasets. The contiguous aware placement algorithm, nevertheless, gives the highest percentage of contiguous memory to VMs.

Consolidation Rate: Here, we define the **consolidation rate** as the ratio of servers used to the total number of servers in the data center. It is important for a new placement algorithm to present a good consolidation rate when compared to existing placement methods. That is, the new placement algorithm should not increase the total number of servers used for a given dataset when compared to existing placement algorithms. Figure 4.7 illustrates the variation in consolidation rate for the MS Azure dataset. We observe that the consolidation rate is similar for the

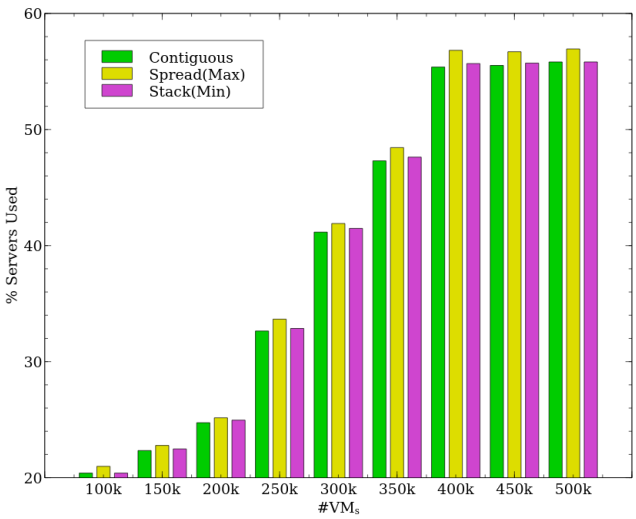

Figure 4.7: Consolidation Rates for the different placement algorithms

different placement algorithms which means our contiguity aware placement algorithm makes little changes to the consolidation rate when compared to traditional placement algorithms.

4.2.4 Analysis of Results

From the different results illustrated above, we observe that for each dataset we have a reasonably high percentage of contiguous memory VMs with the different VM placement algorithms. These high percentages are as a result of the fact that most VM sizes in data centers are small when compared to physical server sizes, as illustrated in most of the datasets. With a small

degree of fragmentation on a physical server, there is therefore a high probability of allocating contiguous machine memory to a virtual machine.

To wrap up this section, we highlight the fact that it is indeed possible to give contiguous memory to virtual machines in today's data centers, as seen by the generally high percentages of contiguous memory VMs in our simulations, and these percentages can be further increased if data centers take into consideration **contiguity-aware VM placement**. Implementing an address translation mechanism based on contiguous memory VMs is therefore feasible and our modifications in Xen hypervisor confirm this as we succeed in allocating contiguous machine memory to both PV and HVM guests.

Conclusion

Review

This work focused on memory virtualization overhead mitigation in virtualized systems. We showed how the overheads of virtualizing memory can be high with state-of-the-art memory virtualization techniques such as nested paging with its slow 2D page walks and shadow paging wherein page table updates cause costly VMM interventions. We proposed a new technique of address translation in virtualized systems which is based on contiguous memory virtual machines, and whose address translation overhead would be theoretically equivalent to base native address translation overhead. We created a software simulator to replay VM traces from various data centers and the results of our simulations show that it is perfectly feasible to allocate contiguous machine memory to VMs using a contiguity-aware placement algorithm. We then modified the Xen hypervisor so it takes into account contiguous memory allocations for newly created VMs. This work would then serve as the basis for the final implementation of our address translation mechanism which is currently being developed by another group of researchers.

Prospects

For future research into this work, we recommend improvements at the simulation and implementation phase. For simulation, larger datasets could be used so as to produce results more coherent with the reality on the ground. Furthermore, we took into account three relatively simple VM placement algorithms in our simulator; Future work on this topic could consider implementing more complex and dynamic virtual machine placement algorithms in the placement simulator. With regard to hypervisor implementations, VMs which do not acquire contiguous machine memory at boot time could have their pages migrated to contiguous regions of memory during idle periods. This would permit the VMs to use our address translation mechanism and thus sustain less memory virtualization overhead.

References

[1] K. Adams and O. Agesen, *A comparison of software and hardware techniques for x86 virtualization*, SIGARCH Comput. Archit. News, 34 (2006), pp. 2–13.

[2] J. Ahn, S. Jin, and J. Huh, *Revisiting hardware-assisted page walks for virtualized systems*, SIGARCH Comput. Archit. News, 40 (2012), pp. 476–487.

[3] H. Alam, T. Zhang, M. Erez, and Y. Etsion, *Do-it-yourself virtual memory translation*, SIGARCH Comput. Archit. News, 45 (2017), pp. 457–468.

[4] P. Barham, B. Dragovic, K. Fraser, S. Hand, T. Harris, A. Ho, R. Neugebauer, I. Pratt, and A. Warfield, *Xen and the art of virtualization*, SIGOPS Oper. Syst. Rev., 37 (2003), pp. 164–177.

[5] T. W. Barr, A. L. Cox, and S. Rixner, *Translation caching: Skip, don't walk (the page table)*, SIGARCH Comput. Archit. News, 38 (2010), pp. 48–59.

[6] R. Bhargava, B. Serebrin, F. Spadini, and S. Manne, *Accelerating two-dimensional page walks for virtualized systems*, SIGPLAN Not., 43 (2008), pp. 26–35.

[7] N. Bhatia, *Performance evaluation of intel ept hardware assist.*, 2009.

[8] A. Bhattacharjee, *Large-reach memory management unit caches*, in Proceedings of the 46th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO-46, New York, NY, USA, 2013, ACM, pp. 383–394.

[9] J. Corbet, *Memory compaction*, 2010.

[10] E. Cortez, A. Bonde, A. Muzio, M. Russinovich, M. Fontoura, and R. Bianchini, *Resource central: Understanding and predicting workloads for improved resource management in large cloud platforms*, in Proceedings of the 26th Symposium on Operating Systems Principles, SOSP '17, New York, NY, USA, 2017, ACM, pp. 153–167.

[11] L. Foundation, *Xen project software overview*.

[12] J. Gandhi, A. Basu, M. D. Hill, and M. M. Swift, *Efficient memory virtualization: Reducing dimensionality of nested page walks*, in Proceedings of the 47th Annual

IEEE/ACM International Symposium on Microarchitecture, MICRO-47, Washington, DC, USA, 2014, IEEE Computer Society, pp. 178–189.

[13] N. T. Hieu, M. D. Francesco, and A. Y. Jääski, *A virtual machine placement algorithm for balanced resource utilization in cloud data centers*, in Proceedings of the 2014 IEEE International Conference on Cloud Computing, CLOUD '14, Washington, DC, USA, 2014, IEEE Computer Society, pp. 474–481.

[14] Intel, *Intel 64 and IA-32 Architectures Optimization Reference Manual*, 01 2011.

[15] D. Klusáček and B. Parák, *Analysis of mixed workloads from shared cloud infrastructure*, in Job Scheduling Strategies for Parallel Processing, D. Klusáček, W. Cirne, and N. Desai, eds., Cham, 2018, Springer International Publishing, pp. 25–42.

[16] B. Loganayagi and S. Sujatha, *Creating virtual platforms for cloud computing*, IEEE International Conference on Computational Intelligence and Computing Research, (2010), pp. 1–4.

[17] M. N. Bennani and D. A. Menasce, *Resource allocation for autonomic data centers using analytic performance models*, in Proceedings of the Second International Conference on Automatic Computing, ICAC '05, Washington, DC, USA, 2005, IEEE Computer Society, pp. 229–240.

[18] B. Pham, V. Vaidyanathan, A. Jaleel, and A. Bhattacharjee, *Colt: Coalesced large-reach tlbs*, in Proceedings of the 2012 45th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO-45, Washington, DC, USA, 2012, IEEE Computer Society, pp. 258–269.

[19] S. Shen, V. v. Beek, and A. Iosup, *Statistical characterization of business-critical workloads hosted in cloud datacenters*, in 2015 15th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing, May 2015, pp. 465–474.

[20] W. Stallings, *Operating Systems: Internal Design and Principles,*, 7 ed., 2011.

[21] D. Sun, G. Chang, Q. Guo, C. Wang, and X. Wang, *A dependability model to enhance security of cloud environment using system-level virtualization techniques*, First International Conference on Pervasive Computing, Signal Processing and Applications (PCSPA), (2010), pp. 305–310.

[22] M. Swanson, L. Stoller, and J. Carter, *Increasing tlb reach using superpages backed by shadow memory*, SIGARCH Comput. Archit. News, 26 (1998), pp. 204–213.

[23] M. Talluri and M. D. Hill, *Surpassing the tlb performance of superpages with less operating system support*, SIGPLAN Not., 29 (1994), pp. 171–182.

[24] D. M. B. THYSTERE, *Optimized resource allocation for the privileged domain in server virtualization*, 2017.

[25] R. UHLIG, G. NEIGER, D. RODGERS, A. L. SANTONI, F. C. M. MARTINS, A. V. ANDERSON, S. M. BENNETT, A. KAGI, F. H. LEUNG, AND L. SMITH, *Intel virtualization technology*, Computer, 38 (2005), pp. 48–56.

[26] VMWARE, *Consolidating mission critical servers*, 2015.

[27] C. A. WALDSPURGER, *Memory resource management in vmware esx server*, SIGOPS Oper. Syst. Rev., 36 (2002), pp. 181–194.

[28] X. WANG, J. ZANG, Z. WANG, Y. LUO, AND X. LI, *Selective hardware/software memory virtualization*, SIGPLAN Not., 46 (2011), pp. 217–226.

Annex

Internship host structure description

IRIT, the Toulouse Computer Science Research Institute, is a joint research unit founded in 1990 through a partnership between Paul Sabatier University, the National Center for Scientific Research (CNRS), the National Polytechnic Institute of Toulouse and the University of Social Sciences of Toulouse, today Toulouse-Capitol University. Michel Daydé has been IRIT's director since July 2011.

Since its inception, it has hosted teams from various university laboratories in Toulouse. It houses about 690 people, including 234 teacher-researchers, 30 researchers, 233 doctoral students, 13 postdocs, and 38 administrative and technical staff (BIATSS). Regarding the number of researchers, IRIT is the largest mixed research unit under the CNRS, as well as the largest French research institute in computer science. Currently, research is organized around seven themes:

1. Analysis and synthesis of information.
2. Indexing and information search.
3. Interaction, Cooperation, and Adaptation through Experimentation.
4. Reasoning and decision.
5. Modeling, algorithms and high-performance computing.
6. Computer architecture, systems and networks.
7. Software Development Security.

Beyond that, IRIT's influence is reflected in various areas at the European and international level, for example, the European laboratory IREP, the French-Spanish Laboratory for advanced studies in information, and perennial cooperation with various countries including the Maghreb, Japan, Armenia, and the Unites States of America.

IRIT has branches in all universities in Toulouse and it works endlessly to establish a continuum of special research services through collaboration and partnerships.

Annex A

```
static int calculate_order(xen_pfn_t total)
{
  unsigned int val = 1;
  int log2;
  while (total >>= 1)
  {
    val <<= 1;
  }
  log2 = -1;
  while (val)
  {
    log2++;
    val >>= 1;
  }
  return log2;
}
```

Listing 1: Function which calculates the corresponding order from the total number of pages

Annex B

```
for (;;)
{
  zone = zone_hi;
  do
  {
    /* Check if target node can support the allocation. */
    if (!avail[node] || (avail[node][zone] < request))
    continue;
    //printk(KERN_WARNING " Zone : %d Zone_Lo : %d  \n",zone,zone_lo);
    /* Find smallest order which can satisfy the request. */
    for (j = order; j <= MAX_ORDER; j++)
    {

      if (!page_list_empty(&heap(node, zone, j)))
      {
        // printk(KERN_WARNING " Page list not empty... ");
        page_list_for_each(pg, &heap(node, zone, j))
        {
          if (page_to_mfn(pg) == next_mfn)
          {
            page_list_del(pg,&heap(node, zone, j));
            goto found;
          }
        }

      }
    }

  } while (zone-- > zone_lo); /* careful: unsigned zone may wrap */
```

Listing 2: Section of heap allocator which allocates contiguous pages

Annex C

```
public void chooseContiguousServer(ArrayList<Server> list) {
 boolean found = false;
 ArrayList<Server> serversOn = new ArrayList<Server>();
 ArrayList<Server> serversOff = new ArrayList<Server>();
 for (int i = 0; i < list.size(); i++) {
   if (list.get(i).hostingVMs.size() != 0) {
     serversOn.add(list.get(i));

   } else {
   serversOff.add(list.get(i));
 }

}

int start = 0;
if (serversOn.size() > 0) {
 found = true;
 Server tempServer = serversOn.get(0);
 for (int i = 0; i < serversOn.size(); i++) {
   tempServer = serversOn.get(i);
   start = tempServer.findBlock(this);
   if (this.contiguousMem) {
     break;

   }

 }
 this.hostingServer = tempServer;

}
if (!found && serversOff.size() > 0) {
 this.hostingServer = serversOff.get(0);

}


if(this.hostingServer!=null){ this.hostingServer.addVM(this);}
else{System.out.println("VM cannot be hosted: "+this.totalMem);return;}
}
```

Listing 3: Contiguity-aware placement algorithm

Annex D

```
public void chooseBiggest(ArrayList<Server> list) {
  int maxIndex = 0;
  double maxMem = 0;

  boolean found = false;
  ArrayList<Server> serversOn = new ArrayList<Server>();
  ArrayList<Server> serversOff = new ArrayList<Server>();
  for (int i = 0; i < list.size(); i++) {
    if (list.get(i).hostingVMs.size() != 0) {
      serversOn.add(list.get(i));

    } else {
    serversOff.add(list.get(i));
  }

}

if (serversOn.size() > 0) {
  found = true;
  for (int i = 0; i < serversOn.size(); i++) {

    if (serversOn.get(i).availableMem > maxMem) {
      maxMem = serversOn.get(i).availableMem;
      maxIndex = i;
    }

  }
  this.hostingServer = serversOn.get(maxIndex);
}

if (!found && serversOff.size() > 0) {
  this.hostingServer = serversOff.get(0);
}

this.hostingServer.addVM(this);


}
```

Listing 4: Max placement algorithm

Annex E

```
  public void chooseSmallest(ArrayList<Server> list) {
    int minIndex = 0;


    boolean found = false;
    ArrayList<Server> serversOn = new ArrayList<Server>();
    ArrayList<Server> serversOff = new ArrayList<Server>();
    for (int i = 0; i < list.size(); i++) {
      if (list.get(i).hostingVMs.size() != 0) {
        serversOn.add(list.get(i));

      } else {
      serversOff.add(list.get(i));
    }

  }

  if (serversOn.size() > 0) {
    double minMem = serversOn.get(0).availableMem;
    found = true;
    for (int i = 0; i < serversOn.size(); i++) {

      if (serversOn.get(i).availableMem < minMem) {
        minMem = serversOn.get(i).availableMem;
        minIndex = i;
      }

    }
    this.hostingServer = serversOn.get(minIndex);
  }
  if (!found && serversOff.size() > 0) {
    this.hostingServer = serversOff.get(0);
  }

  this.hostingServer.addVM(this);


}
```

Listing 5: Min placement algorithm